Automated Rule-Based Compiler Code Optimizer

Phase 1 — Benchmark Dataset Description

A custom benchmark dataset of 100 small programs was constructed to evaluate the six rule-based compiler optimizations implemented in the project. The programs are compatible with the current project lexer/parser and are divided equally across five categories.

Category	Programs	Primary optimization targets
Constant Expressions	20	Constant Folding; Constant Propagation
Algebraic Redundancy	20	Algebraic Simplification; Constant Folding
Dead Code	20	Dead Code Elimination
Common Subexpressions	20	Common Subexpression Elimination
Loop-Based Programs	20	Strength Reduction; Constant Propagation; DCE; Algebraic Simplification
TOTAL	**100**	**All six optimizations represented**

Evaluation design: Each of the 100 programs has five predefined test inputs, giving 500 planned correctness checks. For each check, original and optimized TAC are executed on the same input and their outputs are compared.

Dataset identifiers: C001–C020 (constant), A001–A020 (algebraic), D001–D020 (dead code), E001–E020 (common subexpressions), L001–L020 (loops).

Data source: Custom benchmark dataset constructed specifically for this project. It is not a machine-learning dataset downloaded from Kaggle; the programs are deliberately designed to exercise the compiler optimization techniques under evaluation.